



UT1. PROGRAMACIÓN MULTIPROCESO

ACTIVIDAD 3

Creación y control de procesos en Java

Título: Lanzar subprocessos desde Java y recoger su resultado

Descripción:

Una aplicación real rara vez trabaja sola: lanza otros programas y necesita saber qué le han devuelto. En esta actividad creas un programa Java que lanza a otro como proceso hijo, le pasa datos, lee lo que produce y comprueba si ha terminado bien. Es el mecanismo que usa por debajo cualquier herramienta que ejecuta procesos, como un servidor de compilación.

Objetivos:

- Crear subprocessos desde Java utilizando la clase `ProcessBuilder`.
- Intercambiar información con un subprocesso mediante argumentos y flujos estándar.
- Sincronizar el proceso padre con el hijo y recoger su código de salida.

Instrucciones:

1. Crea el proyecto **A3_SubprocesosJava** y, dentro, la clase **GeneraNumeros**.
 - a. Recibe un argumento con la cantidad de números que debe generar. Si no lo recibe, genera 35.
 - b. Escribe esos números aleatorios entre 0 y 100 por su salida estándar, uno por línea ([documentación java](#) --- [mira el método nextInt](#)).
 - c. Termina con `System.exit(0)` si todo va bien y con `System.exit(2)` si el argumento no es un número.
2. Crea la clase **Lanzador**, que lanza **GeneraNumeros** como proceso hijo.
 - a. Copia estas dos líneas tal cual:

```
String java = System.getProperty("java.home") + "\\bin\\java.exe";  
ProcessBuilder pb = new ProcessBuilder(java, "-cp",  
System.getProperty("java.class.path"), "GeneraNumeros", "50");
```

- b. Lánzalo con `start()` y muestra por pantalla el `pid()` del proceso hijo.

Java no sabe lanzarse a sí mismo: hay que decirle dónde está java.exe y en qué carpeta están tus clases compiladas. Esas dos propiedades del sistema te lo dan hecho. El "50" es el argumento que recibe el hijo.

3. Lee desde el padre lo que escribe el hijo: obtén el flujo con `getInputStream()` envuélvelo en un `BufferedReader` y muestra cada línea precedida de **[HIJO]**



4. Espera al hijo con `waitFor()` y muestra un mensaje distinto según el valor devuelto: 0 correcto, 2 argumento no válido. Prueba también a lanzarlo con el argumento `hola` y captura el resultado.
5. Modifica Lanzador para que lance **dos** procesos `GeneraNumeros` en lugar de uno.
 - a. Lanza los dos y guarda cada Process en una variable distinta.
 - b. Muestra por pantalla el PID de cada uno.
 - c. Lee la salida de los dos.
 - d. Espera a los dos con `waitFor()` y muestra los dos valores devueltos.
 - e. Responde: ¿han generado los dos los mismos números? ¿Comparten memoria? ¿En qué lo notas?
6. Responde en cinco líneas: al ejecutar `Lanzador`, ¿has creado un proceso nuevo o has llamado a un método de tu propio programa? ¿En qué lo has notado?

1. He creado un proceso nuevo en el sistema y no una simple llamada a un método interno.
2. Lo he notado en que Windows le da a cada hijo su propio número de proceso (PID).
3. Además, no comparten memoria y no puedo acceder directamente a sus variables por código.
4. Toda la información tengo que leerla a través de lo que el hijo escribe en la consola.
5. Por último, con `waitFor`, el programa padre tiene que esperar a que el hijo se cierre del todo.

Rúbrica:

0	No se entrega, o lo entregado no corresponde a la actividad.
3	La clase <code>GeneraNumeros</code> funciona por separado, pero <code>Lanzador</code> no llega a crear el proceso hijo.
5	<code>Lanzador</code> crea el proceso hijo con <code>ProcessBuilder</code> y muestra su PID, pero el padre no lee la salida del hijo.
7	El padre lanza un hijo, le pasa el argumento, lee su salida y distingue los códigos de salida, pero no llega a lanzar los dos procesos del apartado 5 o no espera a los dos.
9	Todo lo anterior, con los dos casos del apartado 4 capturados (código 0 y código 2), los dos hijos del apartado 5 funcionando y sus dos PID visibles, y las respuestas de los apartados 5e y 6 justificadas con una evidencia propia: los PID, <code>tasklist</code> o el orden de los mensajes.
10	Además incorpora por iniciativa propia otro mecanismo visto en clase y lo explica: redirección de la salida a un fichero, cambio del directorio de trabajo con <code>directory()</code> o lectura del flujo de error.

Entrega:

- Código fuente de **GeneraNumeros** y **Lanzador** pegado como texto. No se admiten capturas de pantalla del código.

CÓDIGO **GeneraNumeros**:

```
import java.util.Random;

public class GeneraNumeros {
    static void main(String[] args) {
        //Por defecto generamos 35 numeros:
        int cantidad = 35;

        if (args.length > 0) {
            try {
                cantidad = Integer.parseInt(args[0]);
            } catch (NumberFormatException e){
                System.out.println("El argumento no es un numero " + args[0]);
                System.exit(2);
            }
        }

        // Apartado B)
        // Escribe esos números aleatorios entre 0 y 100 por su salida estándar, uno por línea
        Random aleatorio= new Random();

        for (int i=0; i<cantidad; i++) {
            System.out.println(aleatorio.nextInt(101));
        }
        System.exit(0);
    }
}
```

CÓDIGO **Lanzador**:

- Capturas de consola de los apartados 3 y 4, con los dos casos del apartado 4. En las capturas debe verse el autor de la actividad.

```

C:\Users\JGARCIA> java -jar A3_Subprocesos.jar
Autor: Juan Gabriel Salazar Clares
Say el padre, el PID es: 8050
We launched a child PID: 8050
We launched a child PID: 12908

[PID 1] 3
[PID 1] 80
[PID 1] 89
[PID 1] 88
[PID 1] 28
[PID 1] 80
[PID 1] 62
[PID 1] 64
[PID 1] 18
[PID 1] 66
[PID 1] 83
[PID 1] 27
[PID 1] 76
[PID 1] 86
[PID 1] 15
[PID 1] 99
[PID 1] 26
[PID 1] 90
[PID 1] 66
[PID 1] 26
[PID 1] 52
[PID 1] 7
[PID 1] 62

C:\Users\JGARCIA> java -jar A3_Subprocesos.jar 3
Autor: Juan Gabriel Salazar Clares
Say el padre, el PID es: 8050
We launched a child PID: 8050
We launched a child PID: 12908

[PID 1] 3
[PID 1] 80
[PID 1] 89
[PID 1] 88
[PID 1] 28
[PID 1] 80
[PID 1] 62
[PID 1] 64
[PID 1] 18
[PID 1] 66
[PID 1] 83
[PID 1] 27
[PID 1] 76
[PID 1] 86
[PID 1] 15
[PID 1] 99
[PID 1] 26
[PID 1] 90
[PID 1] 66
[PID 1] 26
[PID 1] 52
[PID 1] 7
[PID 1] 62
  
```

```

public class Lanzador {
    public static void main(String[] args) throws Exception {
        System.out.println("He lanzado un hijo 1 PID: " + hijo1.pid());
        System.out.println("He lanzado un hijo 2 PID: " + hijo2.pid());

        // --- LECTURA HIJO 1 ---
        BufferedReader lector1 = new BufferedReader(new InputStreamReader(hijo1.getInputStream()));
        String lineas;
        while ((lineas = lector1.readLine()) != null) {
            System.out.println("[HIJO 1] " + lineas);
        }
    }
}

```

Run: Lanzador

```

[HIJO 2] 99
[HIJO 2] 84
[HIJO 2] 3
[HIJO 2] 84
[HIJO 2] 43
[HIJO 2] 43
[HIJO 2] 26
[HIJO 2] 78
[HIJO 2] 28
[HIJO 2] 34
[HIJO 2] 40
[HIJO 2] 87
[HIJO 2] 81
[HIJO 2] 54
[HIJO 2] 92
[HIJO 2] 38
[HIJO 2] 70
[HIJO 2] 6
[HIJO 2] 69
[HIJO 2] 43
[HIJO 2] 72
[HIJO 2] 70
[HIJO 2] 87
El hijo 1 ha terminado correctamente (codigo 0).
El hijo 2 ha terminado correctamente (codigo 0).
Process finished with exit code 0

```

```

public class Lanzador {
    public static void main(String[] args) throws Exception {
        String java = System.getProperty("java.home") + "\\bin\\java.exe";

        // Preparamos los dos procesos
        ProcessBuilder pb1 = new ProcessBuilder(new String[] { java, "-cp", System.getProperty("java.class.path"), "GeneraNumeros", "hola" });
        ProcessBuilder pb2 = new ProcessBuilder(new String[] { java, "-cp", System.getProperty("java.class.path"), "GeneraNumeros", "hola" });

        // Lanzamos los dos y guardamos cada Process en una variable distinta
        Process hijo1 = pb1.start();
        Process hijo2 = pb2.start();
    }
}

```

Run: Lanzador

```

C:\Users\IEM2\jdk\temurin-25.8.4\bin\java.exe -javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2020.2.2\lib\idea_rt.jar=S4564 -Dfile.encoding=UTF-8
Autor: Juan Gabriel Galarza Claros
Soy el padre, mi PID es: 7560
He lanzado un hijo 1 PID: 7608
He lanzado un hijo 2 PID: 11288
[HIJO 1] El argumento no es un numero hola
[HIJO 2] El argumento no es un numero hola
El hijo 1 ha recibido un argumento no valido (codigo 2).
El hijo 2 ha recibido un argumento no valido (codigo 2).
Process finished with exit code 0

```

- Respuesta escrita del apartado 5.

1. No, cada subprocesso genera una secuencia diferente de números.
2. No, no comparten memoria en absoluto. Cada proceso se ejecuta en su propia instancia aislada de la Máquina Virtual de Java.
3. Se nota porque el ordenador le da un identificador distinto (PID) a cada programa, cada uno saca números diferentes por su cuenta y no pueden pasarse datos de forma directa, sino que el padre tiene que leer lo que el hijo va escribiendo en la pantalla